SCARA · Documento maestro v2

Primera versión para clase · 30/10/2026. Reemplaza como referencia de trabajo al documento maestro v1 (`00-SCARA-Master.pdf`, 02/05/2026), que se conserva sin cambios. Esta versión reúne lo decidido hasta hoy y trae dos propuestas para revisar con el profesor. La versión final sumará la evidencia de la campaña de validación.

Cómo leer el estado de cada parte:

Marca	Significado
Decidido	El grupo lo decidió; está en el registro de decisiones y, si es difícil de revertir, en un ADR.
Propuesta al profesor	La implementamos igual y se revisa en clase. Si el profesor la cambia, se cambia la configuración, no la arquitectura.
Pendiente de evidencia	Lo completa la campaña de validación (ronda de implementación, ticket de campaña).
Parámetro provisional	Valor con fuente o hipótesis explícita, todavía no confirmado. No es una medición.

Contenido

1. Propuestas para revisar en clase (D9, D10/D11)
2. Cambios respecto de la v1 y fuentes
3. Mecánica vigente
4. Transmisión y cinemática híbrida
5. Electrónica
6. Protocolo `scara/1`
7. Homing y seguridad
8. Teach & play
9. Qué se simula y qué no
10. Validación
11. Registro de decisiones

1. Propuestas para revisar en clase

Las dos propuestas ya se están implementando: el profesor puede aceptarlas, corregirlas o pedir otra cosa. Cada una tiene una línea para anotar la respuesta.

P1 · Microstepping 1/8: 1600 pulsos por vuelta (D9)

Qué dice el maestro v1. Los cuatro TB6600 configurados a 400 pulsos por vuelta (medio paso), p. 14.

Qué proponemos. Configurar los TB6600 a 1/8 de paso, 1600 pulsos por vuelta de motor, con sus llaves DIP. El valor es un único parámetro del firmware (`firmware/config.h`, D44), que comparten el Arduino y la planta simulada: cambiarlo no toca la URC ni el protocolo, que habla en unidades articulares (ADR 0003).

Por qué. Con 400 pulsos, un pulso de M1 mueve J1 0,225°. Con el brazo estirado (500 mm del eje de J1 al punto de herramienta) eso es casi 2 mm en el extremo, y un tramo LIN se vería escalonado. Con 1/8 baja a medio milímetro.

Resolución por pulso, con motores de 200 pasos por vuelta (1,8°), reducción 4:1 en J1–J3 y tornillo ACME de 8 mm por vuelta en J4:

Configuración	Pulsos por vuelta de motor	J1–J3 por pulso	Extremo, brazo estirado (500 mm)	Extremo, solo J2 (250 mm)	J4 por pulso
1/2 (maestro v1)	400	0,225°	1,96 mm	0,98 mm	0,020 mm
1/8 (propuesta)	**1600**	**0,05625°**	**0,49 mm**	**0,25 mm**	**0,005 mm**
1/16	3200	0,028°	0,25 mm	0,12 mm	0,0025 mm

Costo: más pulsos por segundo. El UNO genera los pulsos de los cuatro motores. Frecuencias con las velocidades articulares máximas del modelo actual (parámetro provisional: J1 y J2 a 20°/s, J3 a 28,6°/s, J4 a 30 mm/s). M2 suma J1 + J2 por el acople (sección 4), así que puede necesitar el doble que M1.

Configuración	M1	M2 (J1 + J2)	M3	M4	Total
1/2	89 Hz	178 Hz	127 Hz	1500 Hz	≈ 1,9 kHz
1/8	**356 Hz**	**713 Hz**	**509 Hz**	**6000 Hz**	**≈ 7,6 kHz**
1/16	713 Hz	1426 Hz	1019 Hz	12000 Hz	≈ 15 kHz

Como referencia, GRBL genera hasta unos 30 kHz de pulsos en un UNO con interrupción de timer. Con 1/8 queda margen holgado. Con 1/16 el margen se reduce a la mitad sin una mejora visible: el extremo ya se mueve medio milímetro por pulso. La velocidad «1000 rpm» del maestro no es alcanzable con ninguna de las tres: a 1/8 serían 26,7 kHz por motor. Por eso las velocidades se acotan en el firmware.

Lo que no prometemos. El microstepping mejora la suavidad y la resolución ordenada. La exactitud real depende del par por micropaso, de la carga y de las correas. No se presenta 0,5 mm como precisión del robot.

Descartado. 400 pulsos (resolución gruesa para rectas) y 1/16 o más (exige más pulsos de los que conviene pedirle al UNO, sin beneficio visible).

Respuesta del profesor: _____

P2 · Sensores Hall: ubicación y signos de las referencias (D10, D11)

Qué dice el maestro v1. «Sensores de inicio y final de carrera en los cuatro ejes», con imanes de neodimio y sensores de efecto Hall, sin encoders (p. 14). No dice cuántos sensores hay, dónde van ni con qué polaridad; la sección «Sensor Hall» está vacía. El CAD trae un `KY-003` que no está colocado en ningún ensamble y no hay pieza de imán.

Qué proponemos. Un sensor KY-003 por articulación, en la pieza fija de esa articulación, y dos imanes en la pieza que se mueve: uno en el inicio de carrera (referencia de homing) y otro en el fin (protección por sobrerrecorrido). El firmware distingue los dos imanes por el sentido de giro.

Sensor	Fijo en	Imanes en	Mide	Extremo de referencia del homing
H1	Torre	Brazo, cerca de J1 = −135° y +135°	J1	**−135°**
H2	Brazo, en el codo	Antebrazo, cerca de J2 = −135° y +135°	J2 **relativo**	**−135°**, mismo signo que J1
H3	Antebrazo	Polea de muñeca, cerca de J3 = −150° y +150°	J3	Cualquiera de los dos; lo fija la implementación
H4	Cuerpo Z	Pistón, arriba y abajo	J4	**Arriba** (J4 = 0 mm)

Convención de signos: la del modelo actual (`scara/kinematics.py`). J1–J3 son positivos en sentido antihorario visto desde arriba; J4 es positivo hacia abajo. Los límites articulares son parámetros provisionales: ±135° en J1 y J2, ±150° en J3 y 0–120 mm en J4.

Por qué.

- **Un sensor por eje** entra en el Arduino UNO: 15 de 20 pines por USB (sección 5). Dos sensores por eje con entradas propias no entran.
- **H2 en el codo, midiendo J2 relativo.** Los topes que importan son los relativos: cables en los carreteles y choque entre brazo y antebrazo. Medir desde la base (sobre la polea de M2) daría el ángulo absoluto del antebrazo y no protegería esos topes. Los cables de M3 y M4 ya pasan por el brazo, así que llevar uno más al codo no agrega dificultad.
- **J1 y J2 se referencian en el extremo negativo, los dos con el mismo signo.** El homing busca J1 primero y queda en −135°. Si después J2 buscara en +135°, el codo plegado hacia la torre chocaría: con el chequeo de colisiones del repositorio, el soporte Z toca la torre. Con los dos en −135°, el recorrido completo desde la pose de reposo queda libre. Esta conclusión vale con los límites provisionales y las envolventes de colisión simplificadas.
- **J4 arriba** aleja la herramienta de la mesa antes de mover los demás ejes.
- **En Webots**, el imán es un `Emitter` de radio de alcance corto (unos 5 mm) y el KY-003 un `Receiver`. La activación sale de la geometría del modelo, no de leer la posición perfecta del simulador, que es lo que hacía el banco provisional `scara-dev/1`.

Secuencia de homing propuesta (D11). Arranca desde la pose de reposo y termina volviendo a ella.

1. J4 sube hasta su Hall.
2. J1, moviendo M1 y M2 la misma cantidad de pasos: así J2 no cambia mientras se busca J1.

3. J2, con M2 solo: con M1 quieto, M2 cambia únicamente J2.
4. J3, con M3.

En cada eje: avance rápido hasta el Hall, retroceso hasta que se apague y aproximación lenta, siempre desde el mismo lado. La búsqueda está acotada en pasos y en tiempo; si el Hall no aparece o está trabado, el homing aborta sin declarar referencia.

Descartado. Medir J2 desde la base; `TouchSensor` en Webots (genera choques físicos); calcular el Hall con la posición del simulador; buscar J2 en el extremo positivo.

Respuesta del profesor: _____

2. Cambios respecto de la v1 y fuentes

Regla de fuentes. En la clase del 27/10/2026 el profesor indicó que los SolidWorks son la fuente de verdad y que el maestro v1 está desactualizado. Cuando dos fuentes difieren, gana el CAD; cuando el CAD no alcanza, se usa el maestro v1, y la diferencia queda documentada. Ningún dato se presenta como medido: no hay robot físico (ADR 0001).

Tema	Maestro v1 (02/05/2026)	Maestro v2	Origen
Motores	2 NEMA 23 + 2 NEMA 17	Igual. El ensamble guarda un `NEMA23-2` sin ubicación: es una instancia suprimida o residual	D20
Microstepping	400 pulsos/vuelta	1600 pulsos/vuelta, configurable	D9 · propuesta
Sensores Hall	Inicio y fin en los cuatro ejes, sin ubicación	Uno por eje con dos imanes, ubicación y signos definidos	D10, D11 · propuesta
Recorrido de M2	Se trata J2 como M2/4 (p. 8)	M2 fija el ángulo absoluto del antebrazo: J2 = (θM2 − θM1)/4. M2 recorre hasta 540° de salida	Sección 4
Velocidad rotacional	Misma celda con 4,44°/s y 266 rpm (p. 8)	No son equivalentes: 4,44 son pulsos por grado. No se usa esa celda	Transmisión
Protocolo	Capítulo sin contenido	`scara/1`, texto por línea, TCP	D4–D6, D24, D27
Software	Capítulo sin contenido	URC en la PC, firmware C++ único en la UCI	D3, D6
Homing y pose de reposo	No definidos	Secuencia J4, J1, J2, J3; reposo con el brazo estirado	D11, D30

Discrepancias mecánicas todavía abiertas. Las diferencias entre el maestro, los planos sueltos y el CAD (cuerpo Z, pistón, varilla ACME, buje exterior, carreteles A, polea de muñeca, transmisión de M3, ejes de la torre) están en la revisión para el profesor del 27/10/2026 y en Datos para el profesor. Ninguna cambia las propuestas de este documento.

Fuentes. Consigna del TP final (rev. 13/10/2026); maestro v1; planos y ensambles de `Modelos3D/`; clase del 27/10/2026; investigación del 29/10/2026; registro de decisiones D0–D34; ADR 0001–0007; glosario.

3. Mecánica vigente

SCARA de cuatro articulaciones: J1 (hombro), J2 (codo) y J3 (muñeca) rotacionales; J4 prismática vertical.

Dato	Valor	Estado
Distancia J1–J2	250 mm entre ejes	Del CAD
Distancia J2–J3	250 mm entre ejes	Del CAD
Recorrido J1, J2	±135° (270°)	Parámetro provisional, coincide con el maestro

Dato	Valor	Estado
Recorrido J3	±150° (300°)	Parámetro provisional, coincide con el maestro
Recorrido J4	0–120 mm, positivo hacia abajo	Parámetro provisional, coincide con el maestro
Punto de herramienta	Centro de la cara exterior de `Z-SoporteEfector`, sin pinza	Provisional hasta que se defina la herramienta
Envolventes de colisión	Cajas simplificadas por cuerpo rígido	Provisional; cubren ciertos pares, no todos los contactos internos
Masas e inercias	Calculadas del CAD con densidades nominales	Provisional (propiedades)
Sensores Hall	Un KY-003 por articulación, en la pieza fija, y dos imanes en la que se mueve, centrados en los extremos del recorrido, con una zona activa de ±5° (±3 mm en J4). Radios desde el eje: 60, 40 y 30 mm en J1–J3	Propuesta al profesor (D10, D56); en el modelo de Webots desde el issue #8

El montaje del modelo y sus elecciones están en Montaje y autocolisiones y Ensamble CAD.

Hall en Webots (D57). El KY-003 es un `Receiver` y el imán un `Emitter` con su alcance. Webots no entrega paquetes dentro de un mismo robot, así que la planta mide la distancia entre los dos nodos y activa el Hall cuando el imán está más cerca que su alcance. La activación sale de la geometría del modelo: si se corre un imán, se corre el flanco.

El chequeo de colisiones deja libre el camino físico completo del homing y de la vuelta al reposo, medido en Webots (validación del homing). Vale con las envolventes provisionales.

4. Transmisión y cinemática híbrida

Reconstruida de las listas de componentes de los ensambles y de los planos. Certeza alta en la topología y las relaciones; los signos y cruces de correa los fijamos como convención.

- **M1** (base) → polea 20T → correa → 80T sobre el tubo exterior `A-Eje`, abulonado al brazo. Mueve J1 con reducción 4:1.
- **M2** (base) → 20T → 80T sobre el eje interior `A-EjeInterior`, que pasa por dentro del tubo. Arriba lleva una polea 60T; una correa a lo largo del brazo la une con otra 60T sobre `B-Eje`, fijo al antebrazo.
- **M3** (antebrazo) → 20T → 80T en la muñeca. Sin acople.
- **M4** (columna Z) → acople flexible → varilla ACME de 8 mm por vuelta. Sin acople con J3.

Con θ el ángulo del eje de cada motor y los ceros alineados:

```
J1 = θM1 / 4
J2 = (θM2 − θM1) / 4          (porque J1 + J2 = θM2 / 4)
J3 = θM3 / 4
J4 = 8 mm · vueltas de M4
```

Consecuencias que se ven en la simulación:

1. **M2 fija el ángulo absoluto del antebrazo, no J2.** Si gira M1 y M2 queda quieto, el brazo gira, el antebrazo conserva su orientación en el mundo y J2 cambia en sentido opuesto. Es lo que señaló el profesor: mover el motor del hombro mueve hombro y codo.
2. **Para mover solo J1, M1 y M2 dan la misma cantidad de pasos.** El firmware resuelve ese acople; la URC habla en articulaciones (D5).
3. **El rumbo del extremo depende solo de M2 y M3:** $\psi = J1 + J2 + J3 = (\theta M2 + \theta M3)/4$.
4. **M2 recorre más de lo que dice el maestro.** Con J1 y J2 en ±135°, el antebrazo cubre 540° absolutos: 9600 pulsos a 1/8 (2400 a 400 pulsos/vuelta), no 1200.

Conversión a pulsos con 1/8 (6400 pulsos por vuelta de articulación): 17,78 pulsos por grado en J1–J3 y 200 pulsos por mm en J4. Los pulsos de M2 salen de J1 + J2. El firmware lo hace con enteros: 8 pasos cada 45 centésimas de grado y 2 pasos por centésima de mm.

Límites de los motores (D45). La limitación física está en los motores, no en las articulaciones. Se fijan por motor (provisionales hasta tener las curvas de par) y se derivan con el acople: M1 limita a J1, y M2 limita a **J1 + J2**, el ángulo absoluto del antebrazo. Con M1 y M2 a 80 °/s, J1 llega a 20 °/s; si J1 gira a 20 °/s, J2 puede ir de 0 a −40 °/s.

Jog por motor (D46). Un modo de prueba de la URC mueve un solo motor. Es la forma de mostrar el punto 1 en la defensa.

Evidencia (N1 en Webots, 30/10/2026). Un supervisor aparte midió la posición física simulada contra la estimada por pasos (informe):

Orden	Cambio físico simulado
Jog articular de J1	J1 +6,24°; J2, J3 y J4 sin cambio
Jog por motor de M1	J1 +6,24°, J2 −6,24°; rumbo de la herramienta sin cambio
Jog lento de J3	Escalones de 0,05625°: un paso de 1/8

Detalle, tabla por microstepping y contraste con los planos: Transmisión.

5. Electrónica

Componente	Dato	Estado
UCI	Arduino UNO: 2 KB de RAM, 32 KB de flash, 20 pines de E/S, gabinete separado del brazo	Del maestro (D18)
Enlace	USB-serie a 115200 baudios, puenteado a TCP con socat. Alternativa: shield Ethernet	D1, D18
Drivers	Un TB6600 por motor	Del maestro
Microstepping	1/8, 1600 pulsos por vuelta	Propuesta (P1, D9)
M1, M2	Minebea 23KM-K381-P1, NEMA 23, 1,8°	Del maestro; faltan corriente y curva de par
M3	Shinano Kenshi STP-42D221-03, NEMA 17	Del maestro; falta corriente
M4	42D265, NEMA 17, 1,9 V, 0,7 A	Del maestro
Sensores	4 × KY-003 (Hall digital) y 8 imanes	Propuesta (P2, D10)

Componente	Dato	Estado
Alimentación	36 V 10 A para drivers; 12 V 2 A para Arduino y ventiladores	Del maestro

Presupuesto de pines del UNO (D18).

Uso	Pines
STEP + DIR × 4 motores	8
ENABLE común a los cuatro drivers	1
Hall: una entrada por eje	4
Subtotal	**13**
Serie por USB (D0, D1)	2
Total por USB + socat	**15 de 20**
Con shield Ethernet (SPI D10–D13 + D4)	20 de 20, sin margen

Asignación de pines (D47). Deja libres D4 y D10–D13, los del shield Ethernet: pasar a Ethernet no obliga a recablear.

Pines	Uso
D2, D3, D5, D6	STEP de M1–M4 (PORTD: los pulsos de M1 y M2 salen juntos)
A0–A3	DIR de M1–M4
D7	ENABLE común (ENA+ de los TB6600, cátodo común; en alto, sin corriente)
D8, D9, A4, A5	Hall de J1–J4 (KY-003 con pull-up interno; activo en bajo)

Pasos por timer (D48). El núcleo cuenta los pasos; la interrupción del Timer1, a 20 kHz, los convierte en pulsos con los tiempos del TB6600 (DIR estable antes del flanco, 50 µs en alto): hasta 10 000 pasos/s por motor, contra los 6000 que pide J4 a 30 mm/s. Si se juntan pasos pendientes, el firmware espera en lugar de descartarlos.

Compila sin placa. Con `arduino-cli` y el mismo núcleo que corre en Webots: 6544 bytes de flash (20 % de 32 256) y 575 bytes de RAM global (28 % de 2048), sin avisos en el código del proyecto. La prueba automática exige que la RAM global no pase del 75 %, para dejar lugar a la pila.

socat probado con un puerto serie virtual (D49). La URC opera el firmware, compilado para la PC, por un puerto serie virtual y socat, con la misma receta que se usará con el USB del UNO y sin cambios en la URC: arranque con `BOOT`, habilitar, referencia manual, jog de J1 con J2 quieto, deshabilitar, desconexión y nuevo `BOOT` al reabrir el puerto. Receta, cableado y resultados: UCI en el Arduino UNO. Con la placa real no se probó: no hay hardware.

Sin corriente, J4 cae. La varilla ACME de 4 hilos y paso largo probablemente no es autobloqueante. Por eso la pose de reposo tiene J4 abajo, y la falla F2 (driver sin corriente) se demuestra en J4.

6. Protocolo `scara/1`

La URC y la UCI se comunican con un protocolo de texto propio, al nivel de un Arduino. El día del robot real, Webots se reemplaza por el Arduino sin cambiar la URC.

Roles (D1). La URC escucha y la UCI se conecta: el controlador de Webots, socat con el Arduino por USB, o un Arduino con Ethernet. La URC queda siempre esperando; el robot puede reiniciarse o reconectarse sin tocarla. Puerto 9000 para `scara/1` y 9001 para el canal de instrumentación, los dos configurables (D24).

Formato (D4). ASCII, una línea por mensaje terminada en LF (se ignora CR), de hasta 64 bytes, campos separados por espacios y solo enteros. Se puede hablar a mano con `nc` o el monitor serie.

Unidades (D5, ADR 0003). Centésimas de grado en J1–J3 (`9000` = 90,00°) y centésimas de mm en J4. El firmware convierte a pasos y resuelve el acople M1/M2.

Reparto (D6, ADR 0005). La URC resuelve la cinemática inversa, las rectas y el perfil de velocidad, y manda segmentos cortos con duración. El firmware los encola, los ejecuta con su reloj y responde `OK` cuando hay lugar en la cola. Es la división de GRBL y Klipper.

Sentido	Mensaje	Qué hace
URC → UCI	`HELLO`	Pide identificación
URC → UCI	`PING`	Latido para el watchdog
URC → UCI	`ENABLE` / `DISABLE`	Habilita o corta los drivers; la UCI arranca sin corriente y `DISABLE` pierde la referencia (D40)
URC → UCI	`HOME [j]`	Homing de todos los ejes o de uno
URC → UCI	`ZERO`	Referencia manual desde la pose de reposo
URC → UCI	`JOG j ± vel` / `JOG STOP`	Jog articular local (sin referencia)
URC → UCI	`SEG n ms q1 q2 q3 q4`	Encola el segmento n: llegar a esa configuración articular en ms milisegundos
URC → UCI	`STOP` / `HALT`	Frena y conserva la referencia / parada inmediata, que la pierde si había movimiento (D41)
URC → UCI	`STATUS`	Pide un estado inmediato
UCI → URC	`BOOT`	Arranque o reinicio: «no estoy referenciado»
UCI → URC	`HI`	Respuesta a `HELLO`, con versión de firmware y de protocolo
UCI → URC	`S`	Estado, 20 por segundo: modo, ejes referenciados, posición estimada por pasos, bits de Hall, lugares libres en la cola, último segmento terminado

Sentido	Mensaje	Qué hace
UCI → URC	`OK n` / `ERR código palabra`	`OK n`: el segmento n entró en la cola (control de flujo). `ERR`: orden rechazada o evento que frena, con códigos estables. Las demás órdenes no responden si salen bien: el efecto se ve en `S` (D35)
UCI → URC	`EV HOMED j` / `EV LOST j pasos`	Eje referenciado / diferencia de pasos detectada al cruzar un Hall

Valores iniciales (D27). PING cada 250 ms; watchdog de 1 s; segmentos de 20 ms; cola de 16 segmentos (2–3 durante el jog verificado); 115200 baudios. Caben con holgura en un UNO: a 50 segmentos por segundo son unos 2 KB/s, contra unos 11 KB/s del enlace serie.

Canal de instrumentación (D2, ADR 0004). Lo que solo existe en la simulación va por otro puerto: posición física simulada, inyección de fallas, carga y recarga del mundo. Así la URC no puede depender sin querer de información que el robot real no da.

Especificación y conformidad (D42). La especificación completa, con códigos de error, campos de `S` y conversaciones de ejemplo, está en `PROTOCOLO_SCARA_1.md`. Las mismas conversaciones de conformidad se ejecutan como pruebas contra el firmware (compilado en la PC) y contra la URC (con una UCI falsa por TCP), así los dos lados no pueden entender el protocolo de maneras distintas. El ensayo en Webots está en UCI en Webots.

7. Homing y seguridad

Pose de reposo (D30). J1 = J2 = J3 = 0° (brazo estirado) y J4 abajo. Se deja ahí al apagar, con el botón «Estacionar», y desde ahí arranca el homing. Contraída (J2 = ±135°) toca la torre incluso quieta; estirada queda libre y J1 puede barrer hasta los dos extremos.

Homing (D11, D56). Implementado en el issue #8. Antes de empezar, la URC muestra un dibujo de la pose de reposo y pide confirmarla. `HOME` busca J4, J1 (M1 y M2 juntos), J2 (M2 solo) y J3. En cada eje:

1. Si el Hall ya está activo, sale del imán (desde el reposo, J4 descansa sobre el de abajo).
2. Busca a 8 °/s (8 mm/s) hasta que el Hall se activa.
3. Retrocede hasta que se apaga y 3° (3 mm) más.
4. Vuelve a 1 °/s al mismo flanco, que fija la referencia: J1 y J2 en −130°, J3 en −145° y J4 a 3 mm de arriba.

La búsqueda está acotada en recorrido y en 60 s por eje. Un Hall que no aparece (`ERR 11 ausente`) o que no se apaga (`ERR 12 trabado`) aborta el homing sin declarar referencia. Con los cuatro ejes encontrados, la URC vuelve sola a la pose de reposo con un tramo PTP al 50 %, verificado contra choques; «Estacionar» es el mismo tramo.

Paso	J4	J1	J2	J3
Motores	M4	M1 y M2 juntos	M2	M3
Sentido de búsqueda	Arriba	Negativo	Negativo	Negativo
Flanco de referencia	3 mm	−130°	−130°	−145°

Referencia manual. Botón con confirmación que declara referenciada la pose actual, después de llevar el robot con jog hasta las marcas de la pose de reposo. Es menos precisa que el homing.

Sin referencia (D21, D58). El firmware baja todo jog a 5 °/s (5 mm/s). El Hall hace de fin de carrera: un jog que lo activa frena solo (`ERR 10 fin`), y uno que arranca sobre un imán puede salir de él, pero no seguir hacia el tope. Capturar y reproducir quedan bloqueados: `SEG` exige referencia.

Con referencia (D28). La URC planifica también el jog en segmentos cortos, verificados contra límites, alcance y colisiones; el jog frena antes de un límite o de un choque con la torre.

Watchdog. Si la UCI pasa 1 s sin mensajes de la URC, frena sola.

Reinicio de la UCI. El firmware anuncia `BOOT` al arrancar y la URC marca la referencia como perdida. Con USB, abrir el puerto reinicia la mayoría de los Arduino: la URC nunca supone que el robot sigue referenciado tras una reconexión.

Fallas ensayadas (D23).

Falla	Qué es	Cómo se detecta sin encoders
F1	Pérdida de pasos por sobrecarga o aceleración	`EV LOST` al cruzar un Hall
F2	Driver sin corriente	J4 cae; la posición estimada por pasos ya no vale
F3	Hall trabado	El homing aborta o da un cero falso
F4	Desconexión o URC que no responde	Watchdog
F5	Reinicio inesperado de la UCI	`BOOT`: referencia perdida
F6	Correa que patina o se corta	Opcional, si sobra tiempo

El homing desde el reposo y desde un arranque con desvío está ensayado en Webots, y el homing abortado por un Hall trabado, activo o inactivo, en las conversaciones de conformidad del firmware (validación del homing).

> **Pendiente de evidencia:** las fallas F1–F5 en Webots, con capturas y registros de la campaña.

8. Teach & play

El profesor pidió mover el robot a mano, guardar puntos, armar trayectorias con nombre y reproducirlas siguiendo rectas o el menor recorrido. AGSABOT es la referencia de la idea, no de la implementación (D0).

Jog desde la URC (D12, ADR 0007). Con el teclado de la página de la URC, estilo videojuego. Funciona igual con el robot real, que no tiene botonera.

Tecla	Acción
W / S, A / D	Jog cartesiano en el plano
R / F	Subir y bajar
Q / E	Girar la herramienta (rumbo)
Tab	Cambiar a jog articular: las mismas teclas mueven J1–J4
Mayús	Jog lento
Espacio / Retroceso	Capturar el punto / borrar el último
Enter / Esc	Terminar la trayectoria y pedir nombre / frenar

El jog cartesiano nunca cambia la configuración del codo. Desde el brazo estirado, que es una singularidad, el codo dobla hacia el lado preferido: J2 negativo por defecto (D31).

Captura y puntos escritos (D13, D21, D34). Un punto de trayectoria guarda la pose (configuración articular, que incluye la configuración del codo), no el camino. Capturar exige robot referenciado y usa la posición estimada por pasos. También se puede escribir un punto en XYZ y rumbo, o en J1–J4; la URC verifica alcance, límites y choque, y dice qué falla.

Gestor de trayectorias (D29). Lista con XYZ, rumbo, tipo, velocidad y espera; reordenar, borrar, sobrescribir con la pose actual; «Ir al punto» con confirmación; guardar, guardar como, cargar y nueva, con aviso de cambios sin guardar. Las trayectorias se guardan como JSON con nombre, versión de formato, fecha, unidades y puntos.

Tramos (D14, D22). Cada tramo es LIN (el punto de herramienta recorre una recta) o PTP (todas las articulaciones se interpolan y llegan a la vez). Por defecto: LIN, 50 % de la velocidad máxima y espera de 0 s. Velocidad global de 10 % a 100 %, que no se guarda. Antes de cada play se valida la trayectoria completa: alcance, límites, velocidades, colisiones y configuración del codo. Un tramo LIN imposible se rechaza y se ofrece PTP: el robot nunca arranca un tramo que no puede terminar.

Tramo de aproximación y loop (D32, D33). Al apretar play, la URC calcula un tramo PTP desde la pose actual hasta el punto 1, al 25 % × la velocidad global, y muestra su duración. Se omite si el robot ya está ahí. Los parámetros del punto 1 rigen la vuelta del loop, del último punto al primero.

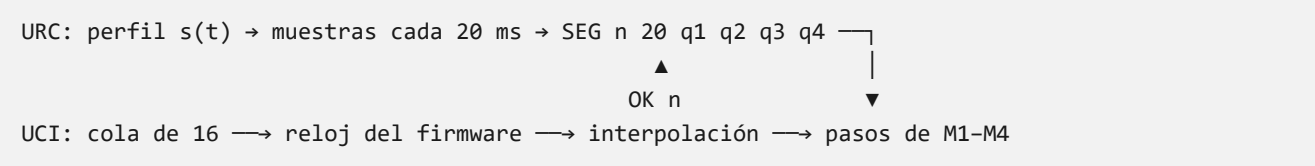
Reproducción (D29). Play desde el principio o desde el punto seleccionado; pausa con rampa y reanudación sin perder la recta del tramo; stop (Esc) que frena y descarta lo pendiente; loop de N veces o infinito; tramo en ejecución resaltado. Los tiempos se respetan aunque la red tenga demoras, porque los ejecuta el reloj del firmware (D6).

Tramo PTP y segmentos temporizados

Implementado en el issue #9. La página de la URC ya lleva el robot a una pose escrita a mano, en J1–J4 o en X, Y, Z y rumbo, con un tramo PTP.

Perfil sincronizado (D53). Todas las articulaciones siguen el mismo perfil de velocidad trapezoidal s(t), de 0 a 1: q(t) = inicio + s(t) · recorrido. Por eso empiezan, aceleran, frenan y llegan a la vez, y el camino es una recta en articulaciones. Los máximos del perfil los fija la restricción más exigente entre las articulaciones del modelo y los motores, con el acople: M2 recorre J1 + J2 (D45). La velocidad del tramo, en porcentaje, reduce las velocidades máximas; la aceleración queda en su máximo.

Segmentos de 20 ms (D27, D50). La URC muestrea el perfil cada 20 ms y envía cada muestra como `SEG n ms q1 q2 q3 q4`. El firmware los encola (16 lugares) y los ejecuta con su reloj, interpolando lineal entre muestras. Un segmento empieza cuando termina el anterior, aunque haya llegado tarde: una demora del enlace de hasta 320 ms no se nota. La URC envía un `SEG` por vez y espera su `OK`. Con la cola llena, el firmware demora el `OK` hasta que se libera un lugar, así la URC nunca envía de más.



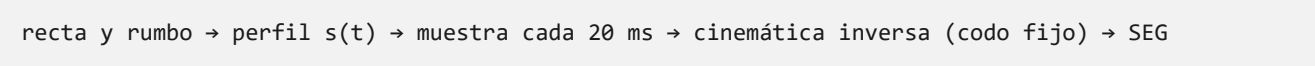
Defensas en el firmware (D51). `SEG` exige corriente y robot referenciado, y no se mezcla con el jog. El firmware verifica que ningún motor supere su velocidad en el segmento (`ERR 8`): protege a los motores de un error de la URC. Aceleraciones, límites y choques los verifica la URC antes de enviar.

Paradas (D52). `STOP` y el watchdog frenan con rampa: cada motor sigue en su sentido y baja la velocidad en línea recta hasta cero, todos en el mismo tiempo, con la aceleración del motor más exigido. Desde 20 °/s, J1 frena en 0,59 s y 5,9°. La cola se vacía y la referencia se conserva. `HALT` para en seco.

Tramo LIN y validación previa

Implementado en el issue #10, como API de Python del planificador (`plan_lin`) y de la validación (`check_lin`, `check_tramo`). El gestor de trayectorias (#12) y la reproducción (#13) la conectan con la página.

Cómo se traza la recta (D54). El punto de herramienta recorre la recta entre los dos puntos, y el rumbo va del de un punto al del otro, con el mismo perfil trapezoidal del PTP. La velocidad del tramo es un porcentaje de 30 mm/s sobre la recta y de 20 °/s de rumbo, que son las velocidades de J4 y de M1. En cada muestra de 20 ms la URC resuelve la cinemática inversa con la configuración del codo del tramo y envía el resultado como un `SEG`.



Cuánto se aparta de la recta. El firmware interpola lineal en articulaciones entre dos muestras, y la URC redondea cada destino a la centésima. El extremo se aparta de la recta a lo sumo **0,069 mm**:

Efecto	Cálculo	Cota
Redondeo a la centésima	J1 y J2 media centésima de grado cada uno, el antebrazo una entera: l1·e + l2·2e	0,066 mm

Efecto	Cálculo	Cota
Cuerda de cada segmento	Brazo y antebrazo giran a lo sumo 0,4° en 20 ms: $(l1 \cdot \Delta 1^2 + l2 \cdot \Delta\varphi^2)/8$	0,003 mm

El plan mide su propio apartamiento: en una recta típica al 50 % es de 0,02 mm. La ejecución suma medio paso de M1 y de M2 (hasta 0,25 mm con 1/8) y el atraso de seguimiento de Webots. Eso lo mide el ensayo.

Qué se verifica antes de arrancar (D14, D55). Cada segmento que se va a enviar, en este orden:

Motivo	Cuándo se rechaza
Límite articular	Un extremo fuera de su recorrido, o el camino lleva J1 o J3 más allá (por ejemplo, al pasar por detrás del robot).
Cambio de codo	J2 cambia de signo entre los extremos. Para cambiar de lado el brazo tendría que estirarse, que es una singularidad.
Fuera de alcance	La recta pasa a menos de 191 mm del eje de J1, el radio mínimo con J2 a ±135°.
Velocidad excesiva	Una articulación o un motor supera su velocidad o aceleración. Pasa cerca del brazo estirado, donde J2 tiene que girar cada vez más rápido para el mismo avance.
Colisión	El chequeo de colisiones existente, en cada segmento. Por ejemplo, una recta entre dos puntos libres que acerca el soporte del efector a la torre.

El rechazo dice el tramo, el motivo y dónde ocurre, y sugiere PTP. Si el motivo es la velocidad, sugiere primero bajarla. Por ejemplo:

```
Tramo 3 (LIN), velocidad excesiva: J2 necesitaría velocidad de 20.24 °/s (máximo 20.05)
al 94 % del tramo, con el brazo casi estirado (J2 = -22.7°). Bajá la velocidad del tramo
o usá un tramo PTP.
```

Desde el brazo estirado, una recta casi siempre se rechaza, aun despacio: J2 tendría que arrancar de golpe. Por eso el primer movimiento de un programa es el tramo de aproximación PTP (D32).

> **Pendiente de evidencia:** teach & play LIN en Webots, con trayectorias de demostración versionadas; desvío de la recta medido contra la posición física simulada. El PTP y los segmentos ya tienen su ensayo en Webots (validación de segmentos).

Captura de puntos y gestor de trayectorias

Implementado en el issue #12, en la URC (`urc/trajectories.py`) y en un panel de su página. La reproducción (#13) lo usa.

Dónde vive la trayectoria (D62). La URC guarda la trayectoria abierta. La página la muestra y pide cada cambio por HTTP. Así, recargar la página no la pierde, y la reproducción planifica desde los mismos datos.

Teclas y lista (D12, D29, D63).

Acción	Cómo
Capturar	Espacio: la posición estimada por pasos, después del punto seleccionado. Exige robot referenciado y quieto.
Borrar	Retroceso: el seleccionado, que después de capturar es el último capturado.
Terminar	Enter: pide el nombre y guarda.
Editar cómo se llega	En cada fila: tipo de tramo (LIN o PTP), velocidad (%) y espera (s). Nacen LIN, 50 % y 0 s (D22).
Reordenar	Arrastrando la fila, o con ↑ y ↓.
Sobrescribir	Reemplaza la pose del seleccionado por la actual; cómo se llega no cambia.
Ir al punto	PTP al 25 %: muestra la duración y mueve recién con la confirmación.

La lista muestra, por punto, X, Y, Z, rumbo y configuración del codo (D31).

Puntos escritos (D34, D63). Se agregan en X, Y, Z y rumbo o en J1–J4. En XYZ el codo es el preferido (J2 negativo) o el elegido; al editar en XYZ, el punto conserva el suyo. Para cambiarlo se edita en J1–J4. Cada punto se valida al capturarlo, escribirlo, editarlo y cargarlo: alcance, límites articulares y choque en reposo. El rechazo dice qué punto falla y por qué:

```
Punto 3: colisión en reposo, <pieza> puede tocar 0-Torre-1 (envolventes y margen de N mm)
```

Los tramos entre puntos se validan antes de cada play (D55), no al editar: dependen del orden y del tipo.

Archivo (D13, D62). Uno por trayectoria, en la carpeta `trayectorias/`, con «Nueva», «Cargar», «Guardar» y «Guardar como». Si hay cambios sin guardar o el archivo ya existe, la URC pregunta. Cada punto guarda las articulaciones en las unidades de `scara/1`, que son las que valen. También guarda el XYZ y el rumbo derivados, para leerlo, y cómo se llega a él:

```
{
  "format": "scara-trayectoria", "version": 1, "name": "Demostración 1",
  "saved": "2026-10-31T12:00:00-03:00",
  "units": {"joints": "centésimas de grado (J1–J3) y de mm (J4)", "tool": "mm (x, y, z) y grados
(rumbo)",
            "speed": "% de la velocidad máxima", "wait": "s"},
  "points": [
    {"joints": [3000, -6000, 1000, 6000], "tool": {"x": 433.01, "y": 0.0, "z": 34.67, "heading":
-20.0},
     "kind": "LIN", "speed": 50, "wait": 0}
  ]
}
```

Al cargar, la URC verifica el formato, la versión, las unidades y cada punto. Un XYZ que no coincide con las articulaciones se rechaza: el archivo se puede editar a mano, pero no se promociona (D34).

9. Qué se simula y qué no

El proyecto simula el diseño; no hay robot físico con el que calibrar (ADR 0001). Cada parámetro declara su procedencia.

Firmware único (D3, ADR 0002). El mismo código C++ corre en el Arduino y dentro del controlador de Webots; solo cambia la capa de hardware. En Webots, la planta simulada reemplaza a drivers, motores, correas y sensores Hall.

Se simula	Cómo
Pasos y microstepping (N1)	Las articulaciones avanzan en pasos discretos
Transmisión y acople M1/M2 (N1)	4:1, 60T–60T al codo, ACME de 8 mm por vuelta; mover M1 cambia J1 y J2
Par limitado y pérdida de pasos (N2)	Con carga o aceleración excesiva el motor salta pasos y el firmware no se entera
Sensores Hall	Imán (`Emitter`) y KY-003 (`Receiver`) del modelo: el Hall se activa por la distancia entre los dos, así que sale de la geometría (D57)
Driver sin corriente	El motor no hace fuerza y sus pasos se pierden; J4 cae por su peso. Al volver la corriente, retiene el eje donde quedó (D60)
Fallas F2–F5	Desde el banco de ensayo, por el canal de instrumentación (D59, D61). F1 llega con N2 (#15)
Tiempo	Webots a 1× durante teach y play; el firmware usa el reloj simulado (D26)
Arranque	Cerca de la pose de reposo, con un desvío aleatorio de ±10° y sin referencia

No se simula	Por qué
Modelo eléctrico de los motores (N3): corrientes, inductancia, resonancias	Faltan datos y exigiría pasos de simulación mucho más chicos (ADR 0006)
Encoders o lazo cerrado de posición	El diseño no los tiene
Pinza o herramienta	Fuera de alcance
Botonera física y teclas en Webots	El jog va por la URC; se puede agregar sin cambiar el firmware
Construcción y cableado del robot real	Se entrega firmware que compila para el UNO y la receta de socat

Canal de instrumentación y banco de ensayo (D2, ADR 0004, D59). Lo que solo existe en la simulación viaja por otro puerto (9001) y otro protocolo de texto, `scara-sim/1`: el controlador de Webots envía la posición física simulada, leída de los sensores de posición de Webots, y recibe las fallas, la carga y la recarga del mundo. En la URC, un panel aparte, «Banco de ensayo (solo simulación)», muestra la columna «Simulado» junto a la estimada por pasos y la diferencia. Sin el canal, como con el robot real, el panel no aparece y la URC funciona igual. Capturar y planificar usan siempre la posición estimada: la sesión, el planificador y la validación no conocen el canal, y una prueba lo verifica.

Falla	Qué hace el banco	Qué se ve
F2	Deja sin corriente el driver de M4	J4 cae a 120 mm en «Simulado»; la estimada no cambia
F3	Traba un Hall activo o inactivo	El homing aborta (`ERR 12` o `ERR 11`) sin referencia
F4	Corta el enlace `scara/1` sin cerrar la conexión	El estado `S` deja de llegar; el watchdog frena el robot
F5	Reinicia el firmware de la UCI	`BOOT` inesperado: sin referencia y sin corriente, así que J4 cae; J1–J3 quedan donde estaban

Datos que faltan para N2. Las curvas de par de los motores que nombra el maestro. Si no se encuentran, se usan valores típicos de NEMA 23 y NEMA 17 declarados como supuestos.

El canal y las fallas F2–F5 están ensayados en Webots (validación del canal de instrumentación).

> **Pendiente de evidencia:** contraste de N2 contra la posición física simulada; curvas de par usadas y su fuente.

10. Validación

> **Pendiente de evidencia.** Esta sección se completa con la campaña de validación nueva (D25). La evidencia del banco provisional `scara-dev/1` queda como registro del proceso y no se cita como prueba del sistema nuevo.

Qué va a traer:

- Pruebas automáticas: firmware por su protocolo en la PC; URC por sus dos bordes (UCI falsa por TCP e interfaz HTTP); planificador como módulo puro.
- Escenarios en Webots: homing desde el reposo; teach & play LIN y PTP; fallas F1–F5; comparación de lo simulado contra lo estimado.
- Compilación en Windows y en Mac (D19).

Ya hay ensayos de la ronda: segmentos y tramo PTP (issue #9) y transmisión N1 (issue #7).

11. Registro de decisiones

Resumen. El texto completo, con qué se descartó y por qué, está en el registro de decisiones.

D	Decisión	Estado	ADR
D0	Teach & play, homing y fallas sobre la simulación existente; AGSABOT como idea	Decidido	
D1	La URC es servidor TCP y la UCI se conecta	Decidido	
D2	Canal de instrumentación aparte	Decidido	0004
D3	Un único firmware C++ para Arduino y Webots	Decidido	0002
D4	Una línea por mensaje, LF, hasta 64 bytes, solo enteros	Decidido	
D5	El protocolo habla en unidades articulares	Decidido	0003
D6	La URC planifica y el firmware ejecuta	Decidido	0005
D7	Nombres UCI y URC	Decidido	
D8	Modelo de motores N1 + N2	Decidido	0006
D9	Microstepping 1/8 (1600 pulsos/vuelta)	**Propuesta al profesor**	
D10	Un Hall por articulación con dos imanes; J2 relativo; referencias J1/J2 en −135°, J4 arriba	**Propuesta al profesor**	
D11	Secuencia de homing J4, J1, J2, J3	Decidido; extremos y signos en la propuesta D10	
D12	Jog con el teclado de la URC	Decidido	0007
D13	Las trayectorias las guarda la URC, como JSON	Decidido	
D14	Tramos LIN o PTP; LIN imposible se rechaza antes de arrancar	Decidido	
D15	Interfaz web local servida por la URC	Decidido	
D16	El camino nuevo se construye al lado del viejo	Decidido	
D17	Maestro v2 en Markdown, exportado a PDF, sin tocar el original	Decidido	
D18	Arduino UNO por USB + socat; un Hall por articulación	Decidido	
D19	Compilación en Windows validada en la bala trazadora	Decidido	
D20	2 NEMA 23 (M1, M2) + 2 NEMA 17 (M3, M4)	Decidido	
D21	Sin referencia, solo jog articular lento	Decidido	
D22	Tramos por defecto: LIN, 50 %, 0 s; velocidad global 10–100 %	Decidido	
D23	Catálogo de fallas F1–F6	Decidido	
D24	Puertos 9000 y 9001; versión en `BOOT` y `HI`	Decidido	
D25	Documentación y evidencia del camino viejo a histórico	Decidido	
D26	Tiempo real durante teach y play	Decidido	
D27	Números iniciales del protocolo	Decidido	

D	Decisión	Estado	ADR
D28	Jog verificado una vez referenciado	Decidido	
D29	Funciones del gestor de trayectorias	Decidido	
D30	Pose de reposo: brazo estirado	Decidido	
D31	Preferencia de configuración del codo: J2 negativo	Decidido	
D32	Tramo de aproximación PTP y vuelta del loop	Decidido	
D33	Tramo de aproximación al 25 %	Decidido	
D34	Puntos definidos sin jog	Decidido	

Las decisiones de implementación de la ronda siguen en el registro desde D35. Las del tramo PTP: D50, un `SEG` sin confirmar por vez y `OK` demorado con la cola llena; D51, el firmware valida la velocidad de cada motor; D52, `STOP` con rampa; D53, perfil PTP trapezoidal común. Las del tramo LIN: D54, velocidad del punto de herramienta y tolerancia sobre la recta; D55, validación previa de cada segmento. Las del homing: D56, homing por flanco en cinco fases y vuelta al reposo planificada por la URC; D57, Hall de Webots por distancia entre imán y sensor; D58, jog lento sin referencia y Hall como fin de carrera. Las del canal de instrumentación: D59, `scara-sim/1` por el puerto 9001; D60, driver sin corriente en la planta; D61, cómo actúan F4, F5 y la carga. Las del gestor de trayectorias: D62, la trayectoria vive en la URC y el archivo `scara-trayectoria` guarda las articulaciones; D63, capturar con el robot referenciado y quieto, codo conservado al editar y validación de cada punto.

ADR 0001 (simular el diseño sin hardware) es anterior al registro y enmarca todo el documento.